Magpie — Sprint 21 spec: lot purchase manager + source ROI

Audience: Codex. **Author:** Claude. **Status: ready to implement. Locks:** D-131 (sequencing — step 5), D-129/D-121 (no AI numbers), the **adopted standards** from Sprint 20 (fee provenance; rankings never recommend loss-making groups). **Facts:** F-09 (no scraping), F-19 (fee model), F-20 (no new eBay API usage). **Builds on:** Sprint 18 (fee engine + all-in cost basis), Sprint 20 (ledger, rankings, FY export), live per `SPRINT20_EVIDENCE.md`.

---

1. Goal

Make **bulk buys honest**. Regan buys estate lots, auction box lots, and bulk deals — one price for many items — but today each `Item` carries an individually fudged `acquisition_cost`, and **every profit number downstream inherits that fudge**. This sprint adds the **Lot** (one purchase event), **human-driven cost allocation** into member items, a **scrapped** path so junk items realise their cost as a loss and a lot's P&L can actually close, **lot-level P&L** ("recovered $X of $Y"), and **Source tagging + source ROI** — completing the dimension Sprint 20 deliberately deferred: *which places you buy from actually make money*.

2. Scope

IN: the **Lot** entity (§3–4) · **allocation helpers + integrity rules** (§5) · **scrapped disposition → realised loss** (§6) · **lot-level P&L view** (§7) · **Source tagging + the source dimension** in ledger/rankings (§8) · **calculator Lot mode** (§9).

OUT (explicitly):

- ❌ Any **automated valuation of lot contents** — allocation is human-driven; helpers assist, humans decide (D-129 spirit). ❌ AI anywhere in these paths. ❌ Scraping / new eBay API calls (F-09 / F-20 gate). ❌ **Retroactive P&L rewrites** — sold items' allocated cost is locked (§5). ❌ Multi-channel (Sprint 22). ❌ Per-item breakdown inside the calculator's Lot mode (v1 is one number for the lot).

3. Data model (additive; backup/restore required)

- **`Lot`** : label, `purchase_date` , **`total_cost`** (all-in for the lot — hammer + buyer's premium + collection postage etc.; a `note` can itemise), `source` (FK, §8), note.
- **`Source`** : name + type (`market` / `estate` / `auction` / `op_shop` / `online` / `private` / `other`).
- **`Item`** additions (all nullable/defaulted): `lot` FK; `source` FK (for **single-item** buys — a lot-linked item inherits the lot's source, don't duplicate); `disposition` (`for_sale` default | `scrapped` , + scrapped date).
- **No parallel cost field:** a lot-linked item's **`acquisition_cost`** **IS its allocated share**. The lot stores the total; integrity = comparing the two (§5). Additive migrations; **pre-migration encrypted backup + restore spot-check** (new tables/fields).

4. The Lot

Create a lot at (or after) purchase; add items to it (new or existing). The lot screen always shows: **total cost · allocated so far · unallocated remainder · members** with their state (unsold / sold / scrapped) and share.

5. Allocation (human-driven, integrity-guarded)

- **Helpers:** (a) **equal split** across current unsold-unlocked members (one tap); (b) **manual per-item** entry with a **live tally — "allocated $X of $Y · remainder $Z"**; (c) **proportional to estimated value** where members have estimates (helper greys out otherwise).
- **The remainder is always visible and honest** — a lot may sit partially allocated; nothing hides it. Negative remainder (over-allocation) is a visible warning state, not silently permitted.
- **Sold-lock (hard rule):** once a member is **sold**, its `acquisition_cost` is **locked** — realised P&L is history and does not rewrite. Re-running a helper redistributes **only among unsold, unlocked members**. (Scrapped members lock at scrap for the same reason.)
- Rounding: allocate to the cent; the helper puts any rounding residue on the last member so the sum is exact (tested).

6. Scrapped → realised loss

Marking a member **scrapped** (junk, unsellable, kept for parts) locks its allocated cost

and **realises it as a loss in the Sprint-20 ledger** on the scrap date (a ledger row with zero revenue, provenance "scrapped"), flowing into the FY export like any other realised outcome. This is what lets a lot's books close truthfully instead of junk cost floating forever.

7. Lot-level P&L

Per lot: total cost · allocated/unallocated · realised revenue + realised profit to date (sold + scrapped members, via the ledger's single profit definition) · remaining cost basis in unsold members · a plain **"recovered $X of $Y"** line. Losses read as losses; a part-allocated lot says so.

8. Source tagging + source ROI

- Tag every **Lot** and every **single-item buy** with a `Source`. Lot members inherit the lot's source.
- The **ledger aggregates and the buy-more ranking gain the source dimension** — same adopted rules: **n ≥ 3 to rank, never recommend loss-making groups**, thin states read thin, tooltip states the rule. This answers "which sources deserve my buying time?" from owned data only.

9. Calculator Lot mode

A **Lot toggle** on the buy calculator: expected **total resale for the lot** (transient what-if, or a figure the human assembles — never persisted as evidence), the usual costs, ROI basis → **Max Buy for the lot** + verdict. Same reverse-margin engine, same shared helpers (no duplicated maths — the parity lesson); labelled clearly as a lot-level number. Per-item breakdown is explicitly v2.

10. Design

Sprint-16 bright high-contrast tokens; phone-first (lots get assessed standing in auction rooms). The allocation tally is impossible to miss; sold/scrapped members visually locked; the lot P&L reads like a ledger card. No overflow at 390 px.

11. Acceptance criteria (closure gate)

- **Allocation:** equal-split / manual-with-live-remainder / proportional helpers correct (tests); **sum-to-the-cent** incl. rounding residue (test); over-allocation warns (test).

- **Sold-lock:** after a member sells, allocation edits/helpers leave its cost untouched and redistribute only among unsold members (test).
- **Scrapped:** realises the allocated cost as a ledger loss on the scrap date, appears in the FY export (tests).
- **Lot P&L:** recovered-vs-total math correct across sold/unsold/scrapped mixes (tests).
- **Source ROI:** ledger + ranking gain the source dimension under **n ≥ 3 + no-loss** rules (tests); lot members inherit the lot's source (test).
- **Lot mode:** correct Max Buy via the shared engine; what-if transient (regression stays green).
- **No new network calls** (assert; F-09/F-20). **No AI in these paths.**
- **Process:** suites + typecheck/build/collectstatic green; **dual-lane Validation green**; **pre-migration encrypted backup + restore spot-check** (new Lot/Source tables + Item fields); Administrator restart (known quirk); **phone screenshots headline** (lot screen with tally, a sold-locked member, the source ranking). Evidence as `SPRINT21_EVIDENCE.md` , **quoting the head commit hash and Validation run ID** in the closure message.

12. For Codex

Reasoning: High. One profit definition — everything flows through the existing fee-engine/ledger helpers; extract, never duplicate. **Hard lines:** allocation is human-driven (no auto-valuation of contents); **sold/scrapped cost is locked — no retroactive P&L rewrites**; the remainder is always visible; rankings keep the adopted **n ≥ 3 + no-loss** standard; no AI numbers (D-129); no scraping / new eBay API (F-09/F-20). Additive migrations with **pre-migration backup + restore spot-check**. **Never echo** `.env` **values** — the credential rotation is still in progress. Evidence as `SPRINT21_EVIDENCE.md` (include the head hash + run ID this time). Next per D-131: Sprint 22 = multi-channel copy pack + manual sold-everywhere board — the chapter's final step.